Git & GitHub Handbook

@aman_views

1. Git Basics & Complete Workflow

Q1. What is Git?

Git is a Distributed Version Control System that helps developers to track changes in code, collaborate with others and manage different versions efficiently.

Q2. Git vs GitHub

GIT	GITHUB
• It is a VCS	• It is a platform
• Works on local system	• Works on cloud
• Command Line Tool	• Web based service
• Tracks changes in files	• Helps in collaboration
• Stores data as snapshots	• Hosts Git repositories

Q3. Why Git?

- Track every change in project
- Work on different versions
- Collaborate in team
- Branching and Merging
- Backup and restore code
- Open source & free

Q4. Complete Git Workflow



Q5. Git Configuration (First Time Setup)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Q6. Most Used Git Commands

Command	Description	Example
git init	Initialize a new Git repository	git init
git status	Check status of working directory	git status
git add <file>	Add file to staging area	git add app.py
git add .	Add all files to staging area	git add .
git commit -m "msg"	Commit staged changes	git commit -m "Initial commit"
git log	Show commit history	git log
git diff	Show changes (unstaged)	git diff
git diff --staged	Show changes (staged)	git diff --staged
git branch	List all branches	git branch
git checkout <branch>	Switch to a branch	git checkout main
git checkout -b <branch>	Create and switch to new branch	git checkout -b feature/login
git merge <branch>	Merge branch into current branch	git merge feature/login
git remote -v	List all remote repositories	git remote -v
git push origin <branch>	Push branch to remote repository	git push origin main
git pull origin <branch>	Pull latest changes from remote	git pull origin main
git clone <url>	Clone remote repository	git clone https://github.com/user/repo.git

Q7. Git File Status

- Untracked - File is new, not tracked by Git.
- Unmodified - File is tracked, no changes.
- Modified - File is changed, not staged.
- Staged - File is staged and ready to commit.
- Committed - File is safely stored in repository.

Q8. .gitignore

Used to ignore files/folders that should not be tracked by Git.

Example (.gitignore)

```
node_modules/
*.log
.env
__pycache__/
```

Git & Github Handbook

2. Branches & Branch Management

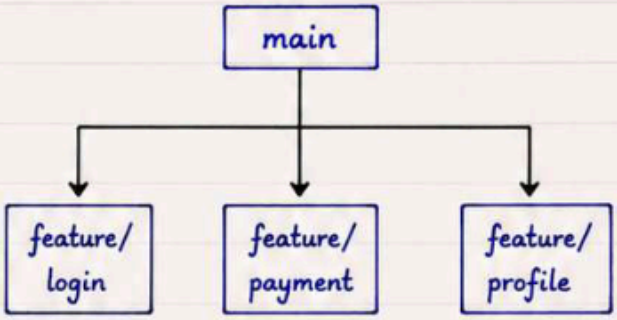
Q1. What is a Branch ?

A branch is a separate line of development. It allows you to work on features without affecting the main code.

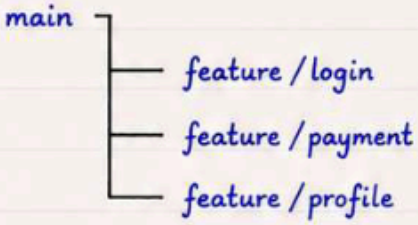
Q2. Branch Commands

- `git branch` → List local branches
  `git branch` / `git branch -v`
- `git branch -a` → List all branches
  `git branch -a`
- `git checkout <branch>` → Switch to branch
  `git checkout feature/login`
- `git switch <branch>` → Switch to branch (new)
  `git switch feature/login`
- `git checkout -b <branch>` → Create & switch
  `git checkout -b feature/payment`
- `git switch -c <branch>` → Create & switch (new)
  `git switch -c feature/profile`
- `git branch -m <new-name>` → Rename branch
  `git branch -m new-name`
- `git branch -d <branch>` → Delete branch (safe)
  `git branch -d feature/login`
- `git branch -D <branch>` → Delete branch (force)
  `git branch -D old-feature`
- `git merge <branch>` → Merge branch
  `git merge feature/login`

Q3. Branch Workflow



Real World Branch Structure



Q4. Example Workflow

```
git checkout -b feature/login     # create branch
git add .                         # stage changes
git commit -m "Login API"         # commit changes
git checkout main                 # switch to main
git merge feature/login           # merge branch
git branch -d feature/login       # delete branch
```

Q5. Quick Summary

Action	Command
List branches	`git branch`
List all branches	`git branch -a`
Switch branch	`git switch <branch>`
Create branch	`git checkout -b <branch>` `git switch -c <branch>`
Rename branch	`git branch -m <new-name>`

Action	Command
Delete branch (safe)	`git branch -d <branch>`
Delete branch (force)	`git branch -D <branch>`
Merge branch	`git merge <branch>`
Check current branch	`git branch --show-current`
Show branch graph	`git log --oneline --graph --all`

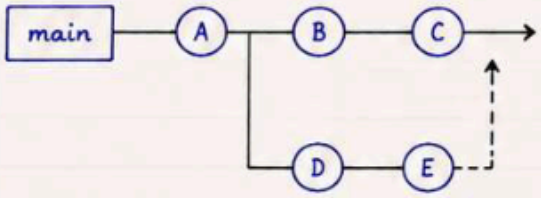
GIT & GITHUB HANDBOOK

@aman_views

3. MERGE, REBASE & CHERRY-PICK

Q1. MERGE

Combine changes from one branch into another.

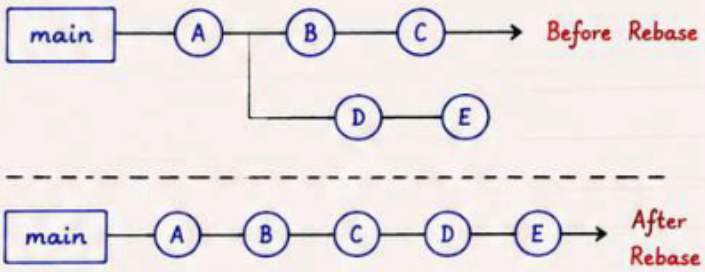


Commands

```
git checkout main
git merge feature/login
```

Q2. REBASE

Move or combine a sequence of commits to a new base.

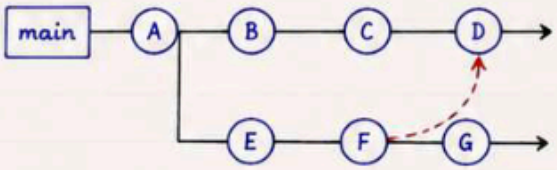


Commands

```
git checkout feature/login
git rebase main
```

Q3. CHERRY-PICK

Apply specific commit(s) from one branch to another.



Commands

```
git checkout main
git cherry-pick <commit-id>
```

Q4. MERGE vs REBASE

Feature	MERGE	REBASE
History	Keeps full history	Creates linear history
Commit ID	New merge commit is created	Old commits are rewritten
Best for	Collaboration (shared branches)	Personal branches / clean history
Risk	Safe	Rewriting history (use carefully)

Q5. WHEN TO USE WHAT?

- Use MERGE when working in a team and want to keep all history.
- Use REBASE when working on your own branch and want a clean, linear history.
- Use CHERRY-PICK when you need only a specific commit, not the whole branch.

Q6. EXAMPLE SCENARIO

· MERGE EXAMPLE

```
git checkout main
git merge feature/login
# brings all changes from
  feature/login to main
```

· REBASE EXAMPLE

```
git checkout feature/login
git rebase main
git checkout main
git merge feature/login
# clean linear history
```

· CHERRY-PICK EXAMPLE

```
git checkout main
git cherry-pick f1a2b3c
# apply only one specific
  commit to main
```

Git & Github Handbook

@aman_views

4. Undo Mistakes (Reset, Revert, Restore, Rm, mv)

Q1. Undo Changes (Not Committed)

- Discard changes in working directory

```
git restore <file>      # undo changes in specific file
git restore .           # undo changes in all files
```

- Unstage changes

```
git restore --staged <file>   # unstage file
git restore --staged .        # unstage all files
```

Q2. Undo Commit

- Undo last commit (keep changes)

```
git reset --soft HEAD~1
```

- Undo last commit (unstage changes)

```
git reset --mixed HEAD~1      # default
```

- Undo last commit (discard changes)

```
git reset --hard HEAD~1       # use carefully!
```

Q3. Reset vs Revert

Feature	RESET	REVERT
What it does	Moves HEAD to previous commit	Creates a new commit that undoes changes
History	Rewrites history	Does not change history
Safe to use on shared branch	No (avoid)	Yes (safe)
Use when	Working locally	Working in team

Q4. Revert Commit

- Create a new commit that undoes changes

```
git revert <commit-id>
```

- Revert last commit

```
git revert HEAD
```

Q5. Remove & Move

- Remove file

```
git rm <file>                 # remove from working dir & staging area
git rm --cached <file>        # remove from staging area only
```

- Move / Rename file

```
git mv old-name new-name      # rename file
```

Q6. Delete Commits Completely

- Delete last N commits

```
git reset --hard HEAD~N
```

Example (N = 3)

```
$ git log --oneline
a1b2c3d (HEAD -> main)  Commit 3
b2c3d4e  Commit 2
c3d4e5f  Commit 1
d4e5f6g  Commit 0
$ git reset --hard HEAD~3    # delete last 3 commits
$ git log --oneline
d4e5f6g (HEAD -> main)  Commit 0
```

Q7. Recover Lost Commits / Changes

- See all reflog

```
git reflog      # shows all actions including deleted commits
```

- Go back to a commit

```
git reset --hard <commit-id>
```

- Recover deleted branch

```
git checkout -b <branch-name> <commit-id>
```

Q8. Summary (Quick Guide)

Task	Command
Undo changes (not staged)	git restore <file> / git restore .
Unstage changes	git restore --staged <file>
Undo last commit (keep changes)	git reset --soft HEAD~1
Undo last commit (unstage changes)	git reset --mixed HEAD~1
Undo last commit (discard changes)	git reset --hard HEAD~1
Revert a commit (safe)	git revert <commit-id>
Remove file	git rm <file>
Rename / Move file	git mv old-name new-name

Git & GitHub Handbook

@aman_views

5. Remote Repository & GitHub Workflow

Q1. Remote Basics

- Remote is a version of your project hosted on the internet (GitHub).
- It allows team collaboration.

Q2. Remote Commands

```
git remote -v                          # list remotes
git remote add origin <url>            # add remote
git remote remove origin               # remove remote
git fetch origin                       # fetch updates
git pull origin <branch>               # fetch + merge
git push origin <branch>               # push changes
git clone <url>                        # clone repo
```

Q3. Clone vs Fetch vs Pull

Command	What it does
git clone	Download entire repo (first time)
git fetch	Download changes only (no merge)
git pull	Download + merge into your branch
git push	Upload local commits to remote

Q4. GitHub Workflow (Day to Day)

1. Clone Repo — `git clone <url>`
2. Create Branch — `git checkout -b feature/login`
3. Code & Commit — `git add .` / `git commit -m "Login API"`
4. Push Branch — `git push origin feature/login`
5. Create Pull Request — Open PR on GitHub
6. Code Review — Team reviews your code
7. Merge PR — Merge to main branch
8. Pull Latest — `git pull origin main`

Q5. Remote Tracking Branches

- Track remote branches for easy collaboration.

```
git branch -r                                  # list remote branches
git branch -a                                  # list all branches
git checkout -b <branch> origin/<branch>       # create local branch from remote
git push -u origin <branch>                    # set upstream & track branch
```

Q6. Upstream vs Origin

Origin	Upstream
• Your fork's remote • Points to your GitHub repo • You push changes here	• Original main repo • Points to original project • You fetch changes from here

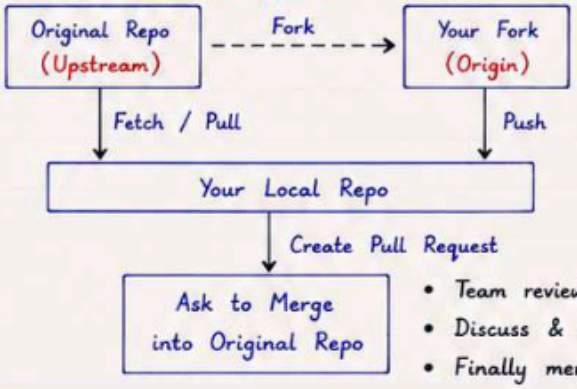
Example

```
# Add upstream (only for fork)
git remote add upstream <original_repo_url>

# Fetch from upstream
git fetch upstream

# Merge upstream/main into your branch
git checkout main
git merge upstream/main
```

Q7. Fork & Pull Request



- Team reviews
- Discuss & approve
- Finally merge

Q8. Useful GitHub Features

Feature	Purpose	Feature	Purpose
Pull Request (PR)	Propose changes & review	Fork	Copy repo to your account
Issues	Track bugs, tasks, features	Watch	Get updates of repo
Projects	Kanban boards for tasks	Star	Bookmark important repos
Actions (CI/CD)	Automate build, test, deploy	Code Review	Review code before merge
Releases	Create versions / releases	Wiki	Document your project

Git & Github Handbook

@aman_views

6. Merge Conflicts & Git Stash

Q1. What is a Merge Conflict ?

A conflict occurs when Git can't automatically merge changes.

Q2. Conflict Example

File: app.txt	
HEAD (main)	feature/login
Line 1 Line 2 Old Code	Line 1 Line 2 New Code

After merge (conflict)

```
<<<<<<< HEAD (main)
Old Code
=======
New Code
>>>>>>> feature/login
```

Fix:

- Edit file and keep the correct code.
- Remove conflict markers.
- Save file and mark as resolved.
- Commit the merge.

```
git add app.txt
git commit
```

Q3. When Conflict Happens ?

- Merging two branches with same changes.
- Pulling latest changes with local changes.
- Rebasing when changes overlap.
- Cherry-pick conflicts.

Q4. Conflict Resolution Steps

1. Git shows conflict in file.
2. Open file and locate conflict markers.
3. Keep the correct changes.
4. Remove <<<<<<<, =======, >>>>>>>.
5. Save file.
6. git add <file>
7. git commit

Q5. Abort Merge / Rebase

- Abort Merge

```
git merge --abort        # back to previous state
```

- Abort Rebase

```
git rebase --abort       # cancel rebase
```

- Continue after resolving conflicts

```
git rebase --continue    # continue rebase
git merge --continue     # continue merge
```

Q6. Git Stash (Save Work Temporarily)

Stash saves your current changes without committing.

Command	Purpose
git stash	Stash local changes
git stash list	List all stashes
git stash pop	Apply latest stash & remove it
git stash apply	Apply latest stash (keep saved)
git stash apply stash@{n}	Apply specific stash
git stash drop	Delete latest stash
git stash clear	Delete all stashes

Q7. Example - Stash Workflow

- You are working on a file but need to switch branch.

```
$ git status              # shows changes
$ git stash               # save changes
$ git switch main         # switch branch
$ git pull                # pull latest changes
$ git switch feature/login  # back to your branch
$ git stash pop           # get your changes back
```

Q8. Conflict Resolution Example

```
1. $ git merge feature/login     # try merge
     (conflict occurs)
2. Edit file -> fix conflict
3. $ git add app.txt             # mark resolved
4. $ git commit                  # complete merge
```

Q9. Quick Summary

Situation	Command
Save work temporarily	git stash
See stash list	git stash list
Apply & remove stash	git stash pop
Apply stash (keep it)	git stash apply
Abort Merge	git merge --abort
Abort Rebase	git rebase --abort
Continue Rebase	git rebase --continue
Continue Merge	git merge --continue

Tips

- Commit small & often.
- Pull latest changes before starting new work.
- Resolve conflicts carefully, never delete code blindly.
- Use stash instead of committing WIP code.

Git & Github Handbook

@aman_views

7. Git Cheat Sheet (Most used)

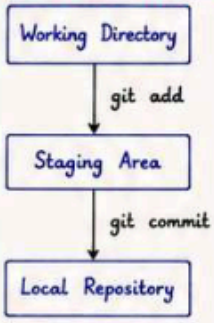
1. Setup & Config

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
git config --list                    # list all config
```

2. Start a Project

```
git init                             # init new repo
git clone <url>                      # clone existing repo
```

3. Basic Workflow




```
git status                           # check status
git add <file>                       # add file
git add .                            # add all
git commit -m "msg"                  # commit
git log --oneline                    # short log
git log --graph
  --oneline --all                    # graph log
```

4. Branching

```
git branch                           # list branches
git branch -a                        # list all branches
git branch <name>                    # create branch
git switch <name>                    # switch branch
git switch -c <name>                 # create & switch
git branch -m <new-name>             # rename branch
git branch -d <name>                 # delete (safe)
git branch -D <name>                 # delete (force)
```

5. Merge, Rebase & Cherry-Pick

```
git merge <branch>                   # merge branch
git rebase <branch>                  # rebase current branch
git rebase --abort                   # abort rebase
git cherry-pick <commit-id>          # pick specific commit
```

Quick Workflow (Day to Day)

1. git pull origin main
2. git switch -c feature/xyz
3. Make changes
4. git add .
5. git commit -m "message"
6. git push -u origin feature/xyz
7. Create PR on GitHub
8. After review → git pull origin main
   git switch main
   git merge feature/xyz

   } keep your main branch updated

6. Remote & Push / Pull

```
git remote -v                        # list remotes
git remote add origin <url>          # add remote
git push -u origin <branch>          # push & set upstream
git push                             # push changes
git pull                             # fetch + merge
git fetch                            # fetch only
git pull --rebase                    # pull with rebase
```

7. Undo & Restore

```
git restore <file>                   # discard changes in file
git restore .                        # discard all changes
git restore --staged <file>          # unstage file
git reset --soft HEAD~1              # undo commit (keep changes)
git reset --mixed HEAD~1             # undo commit (unstage)
git reset --hard HEAD~1              # undo commit (discard)
git revert <commit-id>               # new commit to undo
git rm <file>                        # remove file
git mv old new                       # rename / move file
```

8. Stash

```
git stash                            # stash changes
git stash list                       # list stashes
git stash pop                        # apply & remove stash
git stash apply                      # apply stash (keep it)
git stash drop                       # delete stash
git stash clear                      # delete all stashes
```

9. Most Used Commands (one liners)

- git status - check current status
- git log --oneline --graph - view history
- git diff - see changes
- git show <commit-id> - show commit details
- git branch --show-current - current branch name
- git pull origin <branch> - pull latest
- git push origin <branch> - push your changes
- git remote show origin - remote information
- git clean -fd - remove untracked files & folders

Git Golden Rules

1. Commit often & with meaningful messages.
2. Pull latest changes before starting new work.
3. Use feature branches, not main.
4. Never push code directly to main.
5. Resolve conflicts carefully, test & then commit.